# Codex、Claude Code 与 Open Interpreter CLI 中文手册

适用版本：AI CLI Profile Manager `0.3.19`（source/install/runtime/live 分层验收） 用途：帮助中文用户直接使用原生 Codex CLI、Claude Code 和当前官方 Rust Open Interpreter。

> aicli 只负责选择 Profile 并启动原生 CLI。本手册保留上游英文命令，便于复制和搜索。上游版本会变化；某条命令不在当前 CLI 的 `/help` 或斜杠菜单中时，以当前官方界面为准。

## 1. 最常用命令先看这里

### 1.1 会话内操作对照

| 任务 | Codex CLI | Claude Code | Open Interpreter Rust | 通常何时生效 |
|---|---|---|---|---|
| 查看可用命令 | 输入 `/` 或 `/help` | 输入 `/` 或 `/help` | `/help` | 立即 |
| 切换模型 | `/model` | `/model` | `/model` | 下一请求 |
| 调整思考强度 | `/reasoning`，或在 `/model` 中选择 reasoning effort | `/effort` | 由模型/Provider 配置决定 | 下一请求 |
| 调整权限 | `/permissions` | `/permissions` | `/permissions` | 当前会话 |
| 压缩上下文 | `/compact` | `/compact` | 只在当前版本帮助明确提供时使用 | 当前会话 |
| 查看状态 | `/status` | `/status`（若当前版本提供） | `/help` 查看当前能力 | 立即 |
| 恢复旧会话 | `/resume` 或 `codex resume` | `/resume`、`claude -c` 或 `claude -r` | 以当前 `/help` 为准 | 进入旧会话 |
| 新建会话 | `/new` | `/clear` 或新开进程，以当前版本为准 | 新开 `interpreter` 进程 | 新上下文 |
| 非交互运行 | `codex exec "任务"` | `claude -p "任务"` | `interpreter exec "任务"` | 新进程 |

请勿跨引擎照抄思考命令。当前 Codex 构建可提供独立的 `/reasoning`；若当前斜杠菜单没有该项，就在 `/model` 中选择 reasoning effort。配置层字段是 `model_reasoning_effort`。Claude Code 使用第 3 节所列的独立 effort 命令。

### 1.2 用 aicli 启动

```
aicli start codex-official
aicli start claude-official
aicli start oi-ollama
```

指定项目：

```
aicli start codex-official --project "C:\Work\Project"
aicli start codex-deepseek-flash --project "D:\项目\演示"
```

启动参数在进程创建时生效，会话内斜杠命令在当前会话中生效。需要更换 Provider 时退出当前 CLI，重新运行 `aicli start <Profile ID>`。

程序、benchmark 或 Observer 应使用可恢复 machine 入口，而不是把 `codex exec resume` 字符串塞入新会话：

```
$task | aicli run start codex-ollama-main --stdin --json --project C:\Work\Project --background
aicli run status <run-id> --json
aicli run resume <run-id> --json --background
aicli run abort <run-id> --json
```

`run resume` 调用 app-server `thread/resume`，必须回读原 thread/session 和同一 workspace/Profile/model/provider/effort；新 thread 或任何身份漂移都返回 `resumeSupported=false`。事件和回执按 attempt 追加并链式校验，旧 partial 不得和新 workspace attempt 合并。上层可调用该控制面自动续跑，不需要用户再发一条“继续”消息。

## 2. Codex CLI

### 2.1 模型与思考

```
/model
/reasoning
```

`/model` 用于选择模型，并可在模型支持时选择 reasoning effort；提供 `/reasoning` 的构建也可从独立入口调整。以当前斜杠菜单为准。启动时可透传模型：

```
aicli start codex-official -- --model gpt-5.6-sol
```

第三方 Codex Profile 的 Provider 配置由 aicli 管理。不要透传 `-c`、`--config` 或 `--profile` 覆盖 Provider；这些参数与启动计划冲突，会被拒绝。

DeepSeek Codex 当前使用两个隔离 Profile：`codex-deepseek-flash` 固定官方自动升级模型 ID `deepseek-flash`，`codex-deepseek-v4-pro` 固定 `deepseek-v4-pro` / `DeepSeek-V4-Pro-0813` 且仅供 CLI 使用。两者都是 Responses、1M context、默认用户档 `max`，预设保留默认身份，显式模型参数或独立自定义 Profile 可换同厂商型号，Provider 与 fallback 约束保持；Desktop 只显示“DeepSeek Flash”，不把 V4 Pro 加进模型菜单。AICLI 按厂商引用密码中心已有 Key，不建立 Profile DPAPI 副本；受管配置只写 `env_key`；不要照抄官方示例里的明文 `experimental_bearer_token`。旧 `codex-deepseek` / `deepseek-v4-flash` 已退役并失败关闭。

Qwen 使用两个隔离 exact Codex Profile：`codex-qwen3-7-max-paygo` 固定 `qwen3.7-max-2026-06-08`，`codex-qwen3-8-max-paygo` 固定 `qwen3.8-max-0902`。两者均为北京 Workspace 按量 Responses、983616 context、95% 有效窗口、885254 token（最大上下文 90%）自动压缩阈值，用户 `max` 映射原生最高 `xhigh`。通用 alias、其他快照、preview、Plus、通用 DashScope、Token Plan 与 native model/fallback 参数继续失败关闭。

本机 `Qwen/Qwen3.8-27B` 使用 `codex-ollama-main`，兼容入口 `codex-ollama-qwen3-8-27b` 保留，固定同权重运行标签 `qwen3.8-27b:256k`、Responses、`num_ctx=262144`、`draft_num_predict=0`、`num_batch=128`、`max` 和 no-fallback；OpenCode 对应 `opencode-ollama-qwen3-8-27b`，Desktop 显示 `Qwen3.8 27B MAX (256K)`。1M YaRN 是可选扩展模式，不是 32GB GPU 的默认运行合同。

### 2.2 权限、沙箱和计划模式

```
/permissions
/plan
```

- `/permissions` 调整当前会话的审批模式。
- `/plan` 进入偏规划的工作方式；是否可用以当前版本斜杠菜单为准。
- 交互式 `aicli start` 不替用户改写原生权限。面向程序的 `aicli run` 是另一条明确合同：所有当前及未来 Codex 模型一律在 `thread/start` 与 `turn/start` 选择命名权限 `permissions=:danger-full-access`，同时固定 `approvalPolicy=never`；thread receipt 还必须实际返回 `activePermissionProfile=:danger-full-access` 与 `sandbox.type=dangerFullAccess`。显式请求 `read-only` 或 `workspace-write` 会在模型调用前失败，新增模型也不会获得例外或静默降权。
- 同一 `aicli run` 合同默认在 `thread/start.dynamicTools` 注册受管 `public_web_search`，适用于当前和未来 Codex 模型。它只访问固定 HTTPS RSS provider，拒绝重定向、任意 endpoint/Header/Key，公开事件不含 query/result；`--no-web-search` 只关闭本次搜索，不降低全访问权限。

### 2.3 上下文、状态和用量

```
/compact
/status
/usage
```

- `/compact` 总结已有对话以腾出上下文空间，可能丢失细节。原生 ChatGPT + Codex 保持上游默认；DeepSeek/千问自定义 Provider 通常由客户端本地摘要，不等价于 OpenAI 原生远程压缩，因此不要为了省上下文主动执行 `/compact`。
- `/status` 显示当前模型、权限、可写范围和剩余上下文等会话状态。
- `/usage` 或会话结束页的用量汇总显示上游或客户端统计到的 token / 费用信息；它不是 aicli 的费用计算器，也**不一定**等于云厂商账单。

### 2.4 会话管理

```
/resume
/new
/rename
/archive
/delete
```

- `/resume` 选择已保存对话。
- `/new` 在同一 CLI 中开始新任务。
- `/rename` 为当前任务命名。
- `/archive` 归档但保留记录。
- `/delete` 永久删除会话记录，执行前确认对象。

启动入口：

```
codex resume
codex exec "检查当前项目"
```

### 2.5 MCP、Skills、插件与 Apps

当前 Codex 可在斜杠菜单中提供 `/mcp`、`/skills`、`/plugins`、`/apps` 等入口。它们由 Codex 自身配置，不由 aicli 伪装成跨引擎等价功能。连接 GitHub、Google Drive 等外部服务时，仍需在上游完成插件、App/MCP 和账号授权。

## 3. Claude Code

### 3.1 模型与 effort

```
/model
/effort
```

Claude Code 可在启动时指定：

```
aicli start claude-official -- --model sonnet --effort high
```

第三方 Provider 可能只接受“开/关思考”或自有等级。aicli 不把千问、DeepSeek 的思考语义冒充成官方 Claude 完全等价的 effort。

### 3.2 权限模式

```
/permissions
```

Claude Code 的整会话 permission mode 与工具 Allow/Ask/Deny 规则是两层设置。底栏 `manual mode on` 是默认档，不是没有权限。

默认情况下，`Shift+Tab` 通常循环：

```
manual → acceptEdits → plan
```

启动时指定：

```
# 日常写代码、减少文件编辑确认
aicli start claude-official -- --permission-mode acceptEdits

# 仅适合外部已隔离环境
aicli start claude-official -- --permission-mode bypassPermissions --dangerously-skip-permissions
```

PowerShell 可能吞掉 `--`，导致权限参数未进入 Claude。需要时用：

```
pwsh -NoProfile -File $env:LOCALAPPDATA\aicli\bin\aicli.ps1 --% start claude-official -- --permission-mode acceptEdits
```

`bypassPermissions` 需要新进程启动时明确启用，且通常无法仅靠 `Shift+Tab` 从 `manual` 切到最高档。第三方模型通常不满足 `auto` 模式的模型或账号门槛。

### 3.3 会话与非交互入口

```
claude -c
claude -r
claude -p "检查当前项目"
```

- `-c` / `--continue` 继续当前目录最近的对话。

- `-r` / `--resume` 按会话 ID 或选择器恢复。
- `-p` / `--print` 非交互输出后退出。

会话内常用 `/resume`、`/compact`、`/model`、`/effort`、`/permissions`、`/status` 和 `/logout`；以当前斜杠菜单为准。

### 3.4 第三方 API 常见提示

使用 `claude-custom` 时，上游可能提示检测到自定义 API Key。确认前先用：

```
aicli profile show claude-custom
aicli native claude-custom
```

核对数据去向确实是你选择的 Provider。若同时登录 claude.ai，可能出现双认证或 connectors disabled 提示；这是上游认证优先级提示，不自动代表启动失败。

Claude 的 MCP、插件、Chrome 或外部连接器仍由 Claude Code 自身管理。第三方 API 是否支持对应能力取决于 Provider，不应仅凭按钮存在认定可用。

## 4. Open Interpreter Rust

### 4.1 只支持当前官方 Rust 版

```
irm https://www.openinterpreter.com/install.ps1 | iex
interpreter --version
```

要求输出 `interpreter 0.0.21` 或更高。输出为 `Open Interpreter 0.4.x` 的旧 Python 产品及其安装链和参数体系不属于本产品支持面。

### 4.2 常用操作

```
interpreter
interpreter exec "解释当前目录"
interpreter update
```

会话内使用：

```
/model
/permissions
/help
```

当前 Rust 版采用 Codex 风格 TUI 和 TOML 配置。aicli 使用临时 `-c` 配置 Provider、模型、`base_url` 和 `wire_api`，不会写入旧版 YAML Profile。

### 4.3 权限和代码执行

Open Interpreter 面向本机代码执行。使用 `/permissions` 检查当前审批与沙箱设置；不要把云端模型的输出视为可信命令。

aicli 的公开 Profile：

- 默认沿用上游审批与沙箱设置；可在 `--` 后显式传入 Rust CLI 支持的 `--yolo` 或 `--dangerously-bypass-approvals-and-sandbox`。

- 旧 Python 版 `--auto_run`、`--auto-run`、`-y` 及与 Profile 冲突的参数仍拒绝。
- 云端 Key 只放在目标子进程环境，并从 OI Shell 环境排除。
- Live 文本测试使用只读沙箱、无审批执行工具的禁用路径和临时空目录。

官方资料：快速开始、CLI Reference、配置、Provider。

## 5. 三套 CLI 不等价的地方

| 层面 | Codex | Claude Code | Open Interpreter Rust |
|---|---|---|---|
| 主配置 | `~/.codex/config.toml` | `~/.claude/settings.json` | `~/.openinterpreter/config.toml` |
| aicli Provider 方式 | 独立命名 Profile 文件 | 子进程环境和受控设置 | 临时 `-c` TOML 覆盖 |
| 思考操作 | `/reasoning`，或在 `/model` 中选择 reasoning effort | `/effort` | 取决于模型和 Provider |
| 权限 | `/permissions`、沙箱/审批 | permission mode + `/permissions` | `/permissions` |
| 非交互 | `codex exec` | `claude -p` | `interpreter exec` |
| 本机执行 | Codex 工具 | Claude 工具 | 核心使用场景，需格外注意权限 |

相同名称不保证实现和风险完全相同。切换引擎时先看 `aicli native <Profile ID>`，再看上游当前 `/help`。

## 6. 第一次看懂 CLI（可选学习篇）

本篇故意放在常用命令之后。不读也能按前文使用产品。

### 6.1 终端和项目目录

- **看到什么**：`PS C:\Work\Demo>`
- **意思**：这是当前工作目录；省略 `--project` 时，aicli 会在这里启动 CLI。
- **安全练习**：运行 `Get-Location`。
- **结果**：显示当前文件夹的绝对路径。

项目目录不是“AI 的记忆目录”，而是本次工具读写和 Git 操作的工作根。换项目通常应退出当前 CLI，再到新目录启动。

### 6.2 启动参数和斜杠命令

- **看到什么**：`aicli start claude-official -- --effort high`
- **意思**：第一个 `--` 之前属于 aicli，之后逐项交给 Claude Code。
- **安全练习**：先运行 `aicli native claude-official` 查看基础启动计划。
- **结果**：看到脱敏的可执行文件、环境差异和数据去向。

进入 CLI 后输入 `/model` 属于会话内命令，不会重写 aicli Profile。

### 6.3 请求、会话和进程

| 概念 | 含义 |
|---|---|
| 一次请求 | 你发送一条消息，模型完成一次响应 |

| 概念 | 含义 |
| --- | --- |
| 一个会话 | 多轮共享上下文和会话状态 |
| 恢复会话 | 加载以前保存的会话 |
| 新会话 | 使用干净的对话上下文 |
| 新进程 | 再运行一次 `aicli start`；可换 Provider、项目和启动参数 |

Provider 切换要求新进程；模型、权限等是否能在当前会话变化由各上游 CLI 决定。

### 6.4 模型、思考和权限

- **模型**决定由哪个模型响应。
- **思考强度**调节延迟、token 使用与复杂任务质量；不同 Provider 语义不同。
- **权限**决定工具在改文件、执行命令或访问外部资源前是否询问。
- **计划模式**强调先分析后执行，不等于操作系统级安全沙箱。

安全练习：在一个测试目录中打开 CLI，先查看 `/permissions`，不要直接启用 bypass。

### 6.5 上下文、输入、输出和缓存 token

- **上下文窗口**：模型本次能看到的指令、历史、文件摘要和工具结果总量。
- **input tokens**：本次请求送入模型的内容。
- **output tokens**：模型新生成的内容。
- **cached input tokens**：Provider 复用的输入前缀；是否显示和如何计量由上游决定。
- **remaining context**：当前会话还能容纳多少内容，不等于账号剩余额度。

Token 是计量单位，不等于人民币费用。aicli 不计算费用。Codex 可用 `/status` 或 `/usage` 查看上游提供的信息；其他 CLI 以当前状态页或官方用量页面为准。

#### 第三方模型下 Claude Code 的费用显示

使用 `claude-custom` 等自定义 API 路径时，Claude 欢迎区可能显示 **API Usage Billing** 和 Provider 模型名。会话结束页可能出现类似：

```
Total cost: $1.57 (costs may be inaccurate due to usage of unknown models)
Usage by model:
  third-party-model: … input, … output, … cache read, … cache write
```

如何阅读：

| 内容 | 建议 |
| --- | --- |
| 模型名 | 只作为上游显示；仍以 AICLI Profile 与 Provider 回执核对 exact 身份 |
| input / output token | 可能接近各次 API 返回的 usage 累加，可与百炼监控对照量级 |
| cache read / write | Claude 侧常见统计；是否等于百炼上下文缓存计费口径以阿里云为准 |
| **Total cost 美元金额** | **不可当作阿里云扣款。**脚注 `unknown models` 表示客户端按内置未知模型单价估算，**往往偏高** |
| 改代码行数、API/wall 时长 | 多为会话事实，与账单无关 |

**真费用与真调用量：** 登录阿里云百炼控制台的模型监控 / 账单详情（调用后约一小时更完整）。模型 API Key **不能**替代控制台查询历史账单。aicli 不代查、不代算费用。

### 6.6 压缩为什么会丢细节

`/compact` 会把长历史总结成较短内容。它能腾出上下文，但摘要无法无损保存所有数字、文件细节、授权边界和未完成分支。

- **原生 ChatGPT + Codex**：作为基准，使用上游默认机制，AICLI 不附加压缩限制。
- **Codex + DeepSeek/千问**：自定义 Provider 没有 OpenAI 原生 remote compact；受管 model catalog 负责给出真实窗口，云千问为 983616，本地 `qwen3.6-35b:256k` 与 Qwen3.8-27B 256K 运行标签均为 262144，摘要仍由当前客户端/模型完成。
- **Claude Code + 千问/本地 Qwen/自定义第三方模型**：AICLI 按最终有效模型设置 `CLAUDE_CODE_MAX_CONTEXT_TOKENS` 与 `CLAUDE_CODE_AUTO_COMPACT_WINDOW`。不设置只能提前压缩的百分比覆盖，也不关闭自动溢出保护。未知模型不猜窗口，并清除父进程遗留的窗口/禁压缩变量。
- **OpenCode + 本地千问**：`opencode-ollama-main` 与兼容入口 `opencode-ollama-qwen3-8-27b` 均使用 `qwen3.8-27b:256k`，声明 262144 context/input / 32768 output；两者使用 20000 reserved，保留最近 4 轮/16384 token，并关闭有损 tool-output pruning。一次 `aicli run` 的 checkpoint 不跨 run 持久。

聪明用法是一个会话/run 只做一个内聚里程碑；AICLI 的第三方连续性契约要求在自然边界把目标与验收、约束/授权/owner、规则与关键文件、改动文件及既有脏改动、决定、测试/Live 缺口、阻塞项和下一步写入项目已有 plan/progress/decision。接近真实窗口时优先拆任务或开 fresh session；确需压缩时先落盘。压缩后把摘要当线索，重新读取适用的 `AGENTS.md` / `CLAUDE.md`、状态文档以及 `git status` / `git diff`，再继续编辑；不得建立第二事实源。

### 6.7 Profile、配置、环境变量和官方登录

从稳定到临时大致分为：

1. 上游官方登录和主配置。
2. aicli 随版本发布的 Provider 模板。
3. 你保存的用户 Profile 与凭据引用；需要 Key 的 Profile 一律引用密码中心已登记且带 `native_hosts` 的厂商条目，形式为 `passwordcenter:<vendor>`，不建立本地 DPAPI 副本。
4. 本次启动计划和子进程环境。
5. 会话内模型、权限和临时命令。

aicli 使用子进程隔离，是为了避免 `ANTHROPIC_BASE_URL`、`OPENAI_API_KEY` 等全局变量污染其他终端和官方登录。

旧本地密文只在迁移期兼容读取。用 PowerShell 运行 `Import-Module AiCliProfileManager`，再用 `Invoke-AiCliSecretMigration -DryRun` 预览；去掉 `-DryRun` 执行迁移。迁移按端点主机匹配已登记厂商，回读值一致后才备份并改写 Profile，值不同或未登记时保持原状，旧密文不会删除。需要 Key 的新 Profile 只保存密码中心引用，未登记厂商须先登记。完整操作见《AI CLI Profile Manager 使用手册》的“配置第三方 API”。

### 6.8 从错误回到可验证状态

1. 复制错误中的英文关键词，但不要复制 Key。
2. 运行 `aicli doctor <Profile ID>`。
3. 运行 `aicli profile show <Profile ID>` 核对端点和数据去向。
4. 运行 `aicli native <Profile ID>` 核对启动计划。
5. 静态通过后，才考虑会消耗额度的 `aicli test ... --live`。
6. 上游命令变化时回到官方 `/help` 和本手册链接的官方资料。

## 7. 相关文档

- 产品安装、Profile 和排障：《AI CLI Profile Manager 使用手册》

- 当前验证状态：《兼容性与最终验收状态》
- Codex 官方资料：Codex CLI slash commands
- Claude Code 官方资料：Interactive mode、Permissions、Model configuration
- Open Interpreter 官方资料：Terminal docs